

Estudo de Caso: Rede de Pet Shops

Considere o seguinte cenário:

Uma rede de pet shop, que funciona pela manhã, tarde e noite, atende diversas filiais, oferecendo serviços de estética animal, consultas veterinárias e adestramento. Possui um sistema de informação operacional responsável pelo cadastro de filiais, serviços realizados, funcionários, tutores e seus respectivos animais de estimação.

Os funcionários são cadastrados com as seguintes informações: Matrícula, Nome, Endereço, se for veterinário deve registrar o CRMV, Telefone, Especialidade/Função (Ex: Banho, Tosa, Clínica Veterinária, Adestramento). Os tutores são cadastrados com as seguintes informações: CPF, Nome, Endereço, Telefone e Email. As informações dos animais de cada cliente estão cadastrados como: nome, nome do tutor, espécie, raça, porte, sexo. No início de cada atendimento é cadastrado a situação atual que o animal se encontra, com os seguintes campos: situação no inicial (machucado, doente, com pulgas, carrapatos etc) e situação final.

Cada filial da rede oferece determinados tipos de serviços. Um tipo de serviço pode ser disponibilizado em qualquer filial da rede.

A fim de controlar melhor a rede de pet shops, a Gerência necessita de um módulo de suporte à decisão que possa apresentar indicadores que ajudem a identificar oportunidades de mercado, controlar a produtividade das equipes e direcionar a expansão do negócio.

Após levantamento de requisitos com a Gerência da Empresa, foram identificados os seguintes indicadores de desempenho:

- A. Qual o número de atendimentos por Filial, por período?
- B. Qual o número de atendimentos por Tipo de Serviço?
- C. Qual o número de atendimentos por Turno (Manhã, Tarde, Noite)?
- D. Qual o número de atendimentos por Funcionário?
- E. Quais são os cinco clientes mais atendidos por período ?
- F. Quais são os cinco clientes mais atendidos por filial ? Por qual período ?
- G. Quais os tipos de serviços mais requisitados por Filial?
- H. Qual a distribuição de pets por Espécie (Cão, Gato, etc.)?
- I. Qual a distribuição de atendimentos por Porte do animal (Pequeno, Médio, Grande)?
- J. Quais são as top dez filiais com a maior quantidade de atendimento?
- K. Qual filial possui mais atendimentos para cada serviço (estética, consultas veterinárias e adestramento)?
- L. Qual é o quadro clínico de um animal específico por período?
- M. Quantas vezes um animal voltou com o mesmo quadro clínico ?
- N. Qual foi a evolução clínica de um animal entre os atendimentos ?
- O. Quais doenças são as mais recorrentes ?

Fato Atendimento

Cada registro da tabela fato representa um único atendimento realizado por um determinado funcionário a um animal de estimação (vinculado a um tutor), em uma determinada filial, para um determinado tipo de serviço, em uma data e horário específicos.

dim_tempo

*id_tempo

data

ano

mes

dia

turno

dim_filial

*id_filial

nome

cidade

estado

dim_servico

*id_servico

tipo

categoria

dim_funcionario

*id_funcionario

matricula

nome

endereco

funcao

crm

dim_tutor

*id_tutor

cpf

nome

endereco

telefone
email

dim_pet
*id_pet
nome
especie
porte
sexo

Tb_Atendimento
*id_tempo
*id_filial
*id_serviço
*id_funcionario
*id_tutor
*id_pet
valor
quant_atendimento

/* =====
DEMONSTRAÇÃO DE UM AMBIENTE DE DATA WAREHOUSE

Cenário: Gestão de Atendimentos de Redes de Petshop

Camadas:

- OLTP: ambiente operacional;
- STAGING: área intermediária;
- DW: ambiente dimensional.

Convenções:

- cod_: chave operacional ou chave natural;
- id_: chave substituta, surrogate key;
- data_carga: data de execução do processo ETL.

Dimensões:

- dim_tempo;
- dim_filial;
- dim_servico;
- dim_funcionario;
- dim_tutor;
- dim_pet;

=====

DEMONSTRAÇÃO DE UM AMBIENTE DE DATA WAREHOUSE

Cenário: Gestão de Atendimentos de Redes de Petshop

Camadas:

- OLTP: ambiente operacional;
- STAGING: área intermediária;
- DW: ambiente dimensional.

Convenções:

- cod_: chave operacional ou chave natural;
- id_: chave substituta, surrogate key;
- data_carga: data de execução do processo ETL.

Dimensões:

- dim_tempo;
- dim_filial;
- dim_servico;
- dim_funcionario;
- dim_tutor;
- dim_pet;
- dim_quadro_clinico;

```
/* =====  
DEMONSTRAÇÃO DE UM AMBIENTE DE DATA WAREHOUSE
```

Cenário: Gestão de Atendimentos de Redes de Petshop

Camadas:

- OLTP: ambiente operacional;
- STAGING: área intermediária;
- DW: ambiente dimensional.

Convenções:

- cod_: chave operacional ou chave natural;
- id_: chave substituta, surrogate key;
- data_carga: data de execução do processo ETL.

Dimensões:

- dim_tempo: dimensões estática carregadas por intervalos
- dim_filial;
- dim_servico;
- dim_funcionario;
- dim_tutor;
- dim_pet;
- dim_quadro_clinico;

Granularidade da fato:

- uma linha para cada atendimento.

```
===== */
```

```
/* =====
```

1. CRIAÇÃO DO BANCO DE DADOS

```
=====
```

```
*/
```

```
USE master
```

```
GO
```

```
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name =  
'DW_Atendimentos')
```

```
BEGIN
```

```
CREATE DATABASE DW_Atendimentos;
```

```
END
```

```
GO
```

```
USE DW_Atendimentos;  
GO
```

```
/* =====  
2. CRIAÇÃO DOS SCHEMAS  
===== */
```

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'oltp')  
    EXEC('CREATE SCHEMA oltp');  
GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'staging')  
    EXEC('CREATE SCHEMA staging');  
GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'dw')  
    EXEC('CREATE SCHEMA dw');  
GO
```

```
/* =====  
3. REMOÇÃO DOS OBJETOS ANTERIORES
```

Essa etapa permite executar novamente todo o script.
Os procedimentos são removidos antes das tabelas.

```
===== */
```

```
DROP PROCEDURE IF EXISTS dw.sp_executar_etl;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_fato_atendimento;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_quadro_clinico;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_tempo;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_filial;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_servico;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_funcionario;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_tutor;  
DROP PROCEDURE IF EXISTS dw.sp_carregar_dim_pet;  
DROP PROCEDURE IF EXISTS staging.sp_carregar_staging;  
GO
```

```
DROP TABLE IF EXISTS dw.fato_atendimento;  
DROP TABLE IF EXISTS dw.quadro_clinico;  
DROP TABLE IF EXISTS dw.dim_tempo;  
DROP TABLE IF EXISTS dw.dim_filial;  
DROP TABLE IF EXISTS dw.dim_servico;  
DROP TABLE IF EXISTS dw.dim_funcionario;
```

```
DROP TABLE IF EXISTS dw.dim_tutor;
DROP TABLE IF EXISTS dw.dim_pet;
```

```
DROP TABLE IF EXISTS staging.stg_atendimento;
DROP TABLE IF EXISTS staging.quadro_clinico;
DROP TABLE IF EXISTS staging.stg_filial;
DROP TABLE IF EXISTS staging.stg_tipo_servico;
DROP TABLE IF EXISTS staging.stg_funcionario;
DROP TABLE IF EXISTS staging.stg_tutor;
DROP TABLE IF EXISTS staging.stg_pet;
```

```
DROP TABLE IF EXISTS oltp.atendimento;
DROP TABLE IF EXISTS oltp.quadro_clinico;
DROP TABLE IF EXISTS oltp.funcao;
DROP TABLE IF EXISTS oltp.pet;
DROP TABLE IF EXISTS oltp.tutor;
DROP TABLE IF EXISTS oltp.funcionario;
DROP TABLE IF EXISTS oltp.filial;
DROP TABLE IF EXISTS oltp.endereco;
DROP TABLE IF EXISTS oltp.tipo_servico;
```

```
GO
```

```
/* =====
```

4. AMBIENTE OPERACIONAL - OLTP

```
===== */
```

```
/* Tipos de serviço oferecidos. */
```

```
CREATE TABLE oltp.tipo_servico (
    cod_tipo_servico INT IDENTITY(1,1) PRIMARY KEY,
    nome_tipo_servico VARCHAR(100) NOT NULL
);
```

```
/* Endereços*/
```

```
CREATE TABLE oltp.endereco(
    cod_endereco INT IDENTITY(1,1) PRIMARY KEY,
    cep VARCHAR(8) NOT NULL,
    estado VARCHAR(100) NOT NULL,
```

```

        cidade VARCHAR(100) NOT NULL,
        rua VARCHAR(100) NOT NULL,
        numero INT NOT NULL,
        complemento VARCHAR(100)

);
/* Filiais da Rede. */

CREATE TABLE oltp.filial (
    cod_filial INT IDENTITY(1,1) PRIMARY KEY,
    cod_endereco INT NOT NULL,
    nome_filial VARCHAR(100) NOT NULL,
    CONSTRAINT fk_endereco FOREIGN KEY (cod_endereco) REFERENCES
oltp.endereco(cod_endereco)
);

/* Funcionários da Rede*/

CREATE TABLE oltp.funcionario (
    cod_funcionario INT IDENTITY(1,1) PRIMARY KEY,
    cod_endereco INT,
    matricula INT NOT NULL,
    nome_funcionario VARCHAR(100) NOT NULL,
    CRMV VARCHAR(100) NULL
    CONSTRAINT fk_endereco_funcionario FOREIGN KEY (cod_endereco)
REFERENCES oltp.endereco(cod_endereco)
);

/* Funções de cada funcionário*/
CREATE TABLE oltp.funcao (
    cod_funcao INT IDENTITY(1,1) PRIMARY KEY,
    cod_funcionario INT NOT NULL,
    funcao VARCHAR(100) NOT NULL,
    CONSTRAINT fk_funcao_funcionario FOREIGN KEY (cod_funcionario)
REFERENCES oltp.funcionario(cod_funcionario)
);

/* Os clientes */
CREATE TABLE oltp.tutor(
    cod_tutor INT IDENTITY(1,1) PRIMARY KEY,
    cpf VARCHAR(11) NOT NULL,
    cod_endereco INT NOT NULL,
    nome_tutor VARCHAR(100) NOT NULL,

```

```
email VARCHAR(100) NOT NULL,  
telefone VARCHAR(100) NOT NULL,  
CONSTRAINT fk_endereco_tutor FOREIGN KEY (cod_endereco) REFERENCES  
oltp.endereco(cod_endereco)  
);
```

/* Os animais de cada tutor*/

```
CREATE TABLE oltp.pet(  
cod_pet INT IDENTITY(1,1) PRIMARY KEY,  
cod_tutor INT NOT NULL,  
nome VARCHAR(100) NOT NULL,  
especie VARCHAR(100) NOT NULL,  
porte VARCHAR(100) NOT NULL,  
sexo CHAR(1) NOT NULL  
CHECK (Sexo IN ('F', 'M', 'f', 'm')),  
CONSTRAINT fk_pet_tutor FOREIGN KEY (cod_tutor) REFERENCES  
oltp.tutor(cod_tutor)  
);
```

/* Descrição da situação inicial*/

```
CREATE TABLE oltp.quadro_clinico(  
cod_quadro_clinico INT IDENTITY(1,1) PRIMARY KEY,  
cod_pet INT NOT NULL,  
situacao_inicial VARCHAR(500) NOT NULL,  
situacao_final VARCHAR(500)  
CONSTRAINT fk_pet_quadro_clinico FOREIGN KEY (cod_pet) REFERENCES  
oltp.pet(cod_pet)  
);
```

/*Atendimentos registrados no sistema operacional*/

```
CREATE TABLE oltp.atendimento (  
cod_atendimento INT IDENTITY(1,1) PRIMARY KEY,  
cod_tipo_servico INT NOT NULL,  
cod_filial INT NOT NULL,  
cod_funcionario INT NOT NULL,  
cod_tutor INT NOT NULL,  
cod_pet INT NOT NULL,  
cod_quadro_clinico INT NOT NULL,  
data_inicio DATETIME2 NOT NULL,  
data_fim DATETIME2 NULL,  
prioridade VARCHAR(20) NOT NULL,  
valor DECIMAL(10,2) DEFAULT 0.00,  
CONSTRAINT fk_atendimento_tipo_servico FOREIGN KEY (cod_tipo_servico)  
REFERENCES oltp.tipo_servico(cod_tipo_servico),
```

```

        CONSTRAINT fk_atendimento_filial FOREIGN KEY (cod_filial) REFERENCES
oltp.filial(cod_filial),
        CONSTRAINT fk_atendimento_funcionario FOREIGN KEY (cod_funcionario)
REFERENCES oltp.funcionario(cod_funcionario),
        CONSTRAINT fk_atendimento_tutor FOREIGN KEY (cod_tutor) REFERENCES
oltp.tutor(cod_tutor),
        CONSTRAINT fk_atendimento_pet FOREIGN KEY (cod_pet) REFERENCES
oltp.pet(cod_pet),
        CONSTRAINT fk_atendimento_quadro_clinico FOREIGN KEY
(cod_quadro_clinico) REFERENCES oltp.quadro_clinico(cod_quadro_clinico)
);

```

GO

```

SELECT * FROM oltp.quadro_clinico

```

```

/* =====

```

```

5. DADOS INICIAIS DO AMBIENTE OPERACIONAL (5 REGISTROS)

```

```

===== */

```

-- 1. Endereços

```

INSERT INTO oltp.endereco (cep, estado, cidade, rua, numero, complemento)
VALUES

```

```

('49001000', 'SE', 'Aracaju', 'Av. Hermes Fontes', 100, 'Apto 101'),
('49001001', 'SE', 'Aracaju', 'Rua Bahia', 205, NULL),
('49001002', 'SE', 'Aracaju', 'Av. Ivo do Prado', 50, 'Loja B'),
('49001003', 'SE', 'Aracaju', 'Rua Itabaiana', 800, 'Edifício Comercial'),
('49001004', 'SE', 'Aracaju', 'Av. Augusto Franco', 120, 'Casa');

```

-- 2. Tipos de Serviço

```

INSERT INTO oltp.tipo_servico (nome_tipo_servico) VALUES

```

```

('Banho e Tosa'),
('Consulta Veterinária'),
('Vacinação'),
('Aplicação de Medicamento'),
('Cirurgia');

```

-- 3. Filiais

```

INSERT INTO oltp.filial (cod_endereco, nome_filial) VALUES

```

```

(1, 'Filial Centro'), (2, 'Filial Sul'), (3, 'Filial Norte'), (4, 'Filial Leste'), (5, 'Filial Jardins');

```

-- 4. Funcionários

```

INSERT INTO oltp.funcionario (cod_endereco, matricula, nome_funcionario, CRMV)
VALUES

```

```
(1, 101, 'Ana Paula', NULL),
(2, 102, 'Ricardo Silva', 'CRM-SE 111'),
(3, 103, 'Fernanda Lima', 'CRM-SE 222'),
(4, 104, 'Marcos Oliveira', NULL),
(5, 105, 'Patrícia Rocha', 'CRM-SE 333');
```

-- 5. Funções

```
INSERT INTO oltp.funcao (cod_funcionario, funcao) VALUES
(1, 'Atendente'), (2, 'Veterinário'), (3, 'Veterinário'), (4, 'Auxiliar'), (5, 'Veterinário');
```

-- 6. Tutores

```
INSERT INTO oltp.tutor (cpf, cod_endereco, nome_tutor, email, telefone) VALUES
('11111111111', 1, 'Paulo', 't1@email.com', '79999990001'),
('22222222222', 2, 'Maria', 't2@email.com', '79999990002'),
('33333333333', 3, 'Carlos', 't3@email.com', '79999990003'),
('44444444444', 4, 'Pedro', 't4@email.com', '79999990004'),
('55555555555', 5, 'Bianca', 't5@email.com', '79999990005');
```

-- 7. Pets

```
INSERT INTO oltp.pet (cod_tutor, nome, especie, porte, sexo) VALUES
(1, 'Rex', 'Cachorro', 'Grande', 'M'),
(2, 'Luna', 'Gato', 'Pequeno', 'F'),
(3, 'Thor', 'Cachorro', 'Médio', 'M'),
(4, 'Mia', 'Gato', 'Pequeno', 'F'),
(5, 'Bob', 'Cachorro', 'Pequeno', 'M');
```

-- 8. Quadros Clínicos

```
INSERT INTO oltp.quadro_clinico (cod_pet, situacao_inicial, situacao_final) VALUES
(1, 'Checkup anual', 'Vacinas em dia'),
(2, 'Vômitos constantes', 'Intolerância alimentar'),
(3, 'Ferimento na pata', 'Curativo realizado'),
(4, 'Apatia', 'Vitaminas prescritas'),
(5, 'Limpeza de ouvidos', 'Procedimento concluído');
```

-- 9. Atendimentos

```
INSERT INTO oltp.atendimento (cod_tipo_servico, cod_filial, cod_funcionario,
cod_tutor, cod_pet, cod_quadro_clinico, data_inicio, data_fim, prioridade, valor)
VALUES
(1, 1, 1, 1, 1, 1, '2026-07-16 08:00:00', '2026-07-16 09:00:00', 'Baixa', 90.00),
(2, 2, 2, 2, 2, 2, '2026-07-16 09:00:00', '2026-07-16 10:00:00', 'Alta', 60.00),
(5, 3, 3, 3, 3, 3, '2026-07-16 10:00:00', '2026-07-16 12:00:00', 'Alta', 40.00),
(3, 4, 5, 4, 4, 4, '2026-07-16 14:00:00', '2026-07-16 14:30:00', 'Média', 50.00),
(1, 5, 1, 5, 5, 5, '2026-07-16 15:00:00', '2026-07-16 16:00:00', 'Baixa', 30.00);
GO
```

```

SELECT * FROM oltp.quadro_clinico
SELECT * FROM oltp.tipo_servico
SELECT * FROM oltp.endereco
SELECT * FROM oltp.filial
SELECT * FROM oltp.funcionario
SELECT * FROM oltp.funcao
SELECT * FROM oltp.tutor
SELECT * FROM oltp.pet
SELECT * FROM oltp.atendimento

```

```

/* =====

```

6. ÁREA DE STAGING

A staging armazena uma fotografia dos dados operacionais para cada data de carga.

Ao reprocessar uma data:

- somente os registros daquela data são removidos;
- as demais cargas são preservadas.

```

===== */

```

```

CREATE TABLE staging.stg_tipo_servico (
    cod_tipo_servico INT NOT NULL,
    nome_tipo_servico VARCHAR(100) NOT NULL,
    data_carga DATE NOT NULL
);

```

```

CREATE TABLE staging.stg_filial (
    cod_filial INT NOT NULL,
    cod_endereco INT NOT NULL,
    nome_filial VARCHAR(100) NOT NULL,
    data_carga DATE NOT NULL
);

```

```

CREATE TABLE staging.stg_funcionario (
    cod_funcionario INT NOT NULL,
    cod_endereco INT,
    matricula INT NOT NULL,
    nome_funcionario VARCHAR(100) NOT NULL,
    CRMV VARCHAR(100) NULL,
    data_carga DATE NOT NULL
);

```

```
CREATE TABLE staging.stg_tutor(  
    cod_tutor INT NOT NULL,  
    cpf VARCHAR(11) NOT NULL,  
    cod_endereco INT NOT NULL,  
    nome_tutor VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    telefone VARCHAR(100) NOT NULL,  
    data_carga DATE NOT NULL  
);
```

```
CREATE TABLE staging.stg_pet(  
    cod_pet INT NOT NULL,  
    cod_tutor INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    especie VARCHAR(100) NOT NULL,  
    porte VARCHAR(100) NOT NULL,  
    sexo CHAR(1) NOT NULL,  
    data_carga DATE NOT NULL  
);
```

```
CREATE TABLE staging.stg_quadro_clinico(  
    cod_quadro_clinico INT NOT NULL,  
    cod_pet INT NOT NULL,  
    situacao_inicial VARCHAR(500) NOT NULL,  
    situacao_final VARCHAR(500),  
    data_carga DATE NOT NULL  
);
```

```
CREATE TABLE staging.stg_atendimento (  
    cod_atendimento INT NOT NULL,  
    cod_tipo_servico INT NOT NULL,  
    cod_filial INT NOT NULL,  
    cod_funcionario INT NOT NULL,  
    cod_tutor INT NOT NULL,  
    cod_pet INT NOT NULL,  
    cod_quadro_clinico INT NOT NULL,  
    data_inicio DATETIME2 NOT NULL,  
    data_fim DATETIME2 NULL,  
    prioridade VARCHAR(20) NOT NULL,  
    valor DECIMAL(10,2) DEFAULT 0.00,  
    data_carga DATE NOT NULL  
);  
GO
```

/* Índices para facilitar a localização dos dados de uma carga. */

```
CREATE          INDEX          ix_stg_atendimento_data_carga          ON
staging.stg_atendimento(data_carga);
CREATE          INDEX          ix_stg_quadro_clinico_data_carga          ON
staging.stg_quadro_clinico(data_carga);
CREATE INDEX ix_stg_filial_data_carga ON staging.stg_filial(data_carga);
CREATE          INDEX          ix_stg_tipo_servico_data_carga          ON
staging.stg_tipo_servico(data_carga);
CREATE          INDEX          ix_stg_funcionario_data_carga          ON
staging.stg_funcionario(data_carga);
CREATE INDEX ix_stg_tutor_data_carga ON staging.stg_tutor(data_carga);
CREATE INDEX ix_stg_pet_data_carga ON staging.stg_pet(data_carga);
GO
```

/* =====

7. AMBIENTE DIMENSIONAL

===== */